

Start Day 4

Contents

01  CivicFlow-AI — Day 4 Complete Interview Notes D... RESPONSE

CivicFlow-AI — Day 4 Complete Interview Notes

Day 4 was one of the most important days of your CivicFlow-AI project because you moved from **individual AI components** to a connected **Incident Intelligence pipeline**.

You worked on:

1. Priority Engine
2. AI Classification + Priority Integration
3. Embedding Integration
4. Semantic Duplicate Detection
5. Duplicate Prevention
6. HTTP 409 Conflict
7. Service Orchestration
8. Retry & Exponential Backoff
9. RAG + Incident Intelligence
10. RAG Context and Sources
11. Error Handling for External AI Services
12. Complete End-to-End Architecture

1. Day 4 Big Picture

Before Day 4, your system had separate components:

```
text

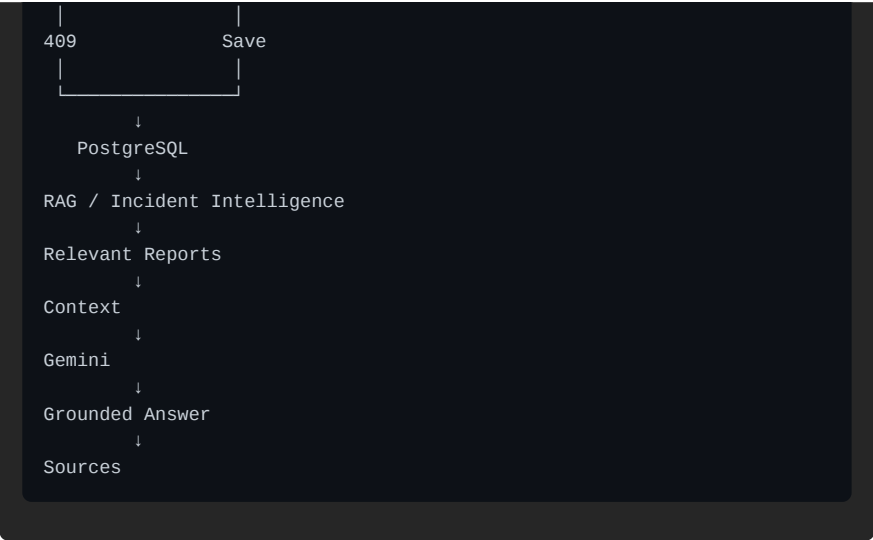
AI Classification
Embedding
Similarity Search
RAG
PostgreSQL
```

The goal of Day 4 was to connect them.

Your final architecture became:

```
text

Citizen Complaint
  ↓
FastAPI
  ↓
Report Service
  ↓
AI Classification
  ↓
Severity
  ↓
Priority Engine
  ↓
Embedding Generation
  ↓
Semantic Duplicate Detection
  ↓
┌───────────┴───────────┐
Duplicate             New
```



2. Priority Engine

What is a Priority Engine?

The Priority Engine is a deterministic business-rule component that converts **severity** into an operational **priority**.

Your implementation:

```
python
def calculate_priority(severity: str) -> str:
    severity = severity.upper()

    if severity == "CRITICAL":
        return "URGENT"
```

```
if severity == "HIGH":
    return "HIGH"

if severity == "MEDIUM":
    return "MEDIUM"

return "LOW"
```

The mapping is:

Severity	Priority
Critical	URGENT
High	HIGH
Medium	MEDIUM
Low	LOW

Why did you create a separate Priority Service?

Because priority is a **business rule**, not something you should blindly trust from the LLM.

The LLM can classify:

```
text

severity = Critical
```

Then your backend determines:

```
text

Critical → URGENT
```

This makes the operational decision deterministic.

Interview question

Why don't you directly use the priority returned by Gemini?

Answer:

"I don't directly trust the LLM-generated priority for an operational business rule. I use the AI-generated severity as an input and calculate the final priority using deterministic backend rules. This makes the priority logic predictable, testable, and easier to change independently of the AI model."

3. Case Normalization

You encountered an important problem.

Initially:

```
python

if severity == "Critical":
```

worked for:

```
text

Critical
```

but failed for:

```
text

CRITICAL
```

because Python string comparisons are case-sensitive.

You solved it using:

```
python

severity = severity.upper()
```

Now:

```
text

Critical → CRITICAL
CRITICAL → CRITICAL
critical → CRITICAL
```

all behave consistently.

Interview question

Why did you normalize the severity string?

Answer:

"LLM output is probabilistic and string formatting can vary. I normalize the severity before applying business rules so that capitalization differences don't cause incorrect priority calculations."

4. AI Classification + Priority Integration

Your report pipeline became:

```
text
  ↓
Complaint
  ↓
classify_report()
  ↓
AIReportResponse
  ↓
severity
  ↓
calculate_priority()
  ↓
final priority
```

For example:

```
text
  ↓
Complaint:
Major water pipeline burst near Ward 5
```

```
AI:
category = Water
severity = Critical
summary = ...
```

```
Backend:
priority = URGENT
```

Your successful report demonstrated:

```
json
{
  "category": "Water",
  "severity": "Critical",
  "priority": "URGENT"
}
```

5. Why Separate AI Logic and Business Logic?

This is a very important interview concept.

Your architecture separates:

```
text
  ↓
ai_service.py
```

from:

```
text

priority_service.py
```

AI Service

Responsible for:

- Gemini communication
- Prompt
- Classification
- Structured AI response
- AI errors

Priority Service

Responsible for:

- Business rules
- Severity normalization
- Priority calculation

Benefit

You can change:

```
text

Gemini model
```

without changing:

```
text

Priority rules
```

And you can change:

```
text

Priority rules
```

without changing:

```
text

Gemini integration
```

Interview answer

"I separated AI responsibilities from deterministic business logic. The AI service handles model interaction and classification, while the priority service handles operational business rules. This improves maintainability, testability, and separation of concerns."

6. Embedding Generation

Day 4 reused the embedding functionality you built on Day 3.

The complaint description is converted into an embedding:

```
text
Text
↓
Embedding Model
↓
Vector
```

Conceptually:

```
text
"A water pipeline has burst..."
↓
[0.012, -0.231, 0.451, ...]
```

Your CivicFlow system uses embeddings for:

- Semantic duplicate detection
- Similarity search
- RAG retrieval
- Incident intelligence

7. Semantic Duplicate Detection

This was one of the major Day 4 features.

Your service:

```
python
def check_duplicate_report(
    db: Session,
    embedding: list[float],
    threshold: float = 0.20
):
```

retrieves the most similar existing report.

The query uses:

```
python
Report.embedding.cosine_distance(embedding)
```

8. Cosine Distance

Your system uses **cosine distance**.

Important rule:

```
text
Smaller distance
↓
More semantically similar
```

For example:

```
text
```

```
0.008
```

is much more similar than:

```
text
```

```
0.50
```

Your configured threshold is:

```
text
```

```
0.20
```

Therefore:

```
text
```

```
distance <= 0.20
```

```
↓
```

```
duplicate
```

and:

```
text
```

```
distance > 0.20
```

```
↓
```

```
not duplicate
```

9. Why Semantic Duplicate Detection?

Exact text matching would require:

```
text
```

```
Complaint A:  
Water pipeline has burst near Ward 5.
```

and:

```
text
```

```
Complaint B:  
A major pipeline is leaking and flooding Ward 5.
```

to be exactly the same to detect duplication.

That's not practical.

Embeddings allow the system to compare **semantic meaning** rather than exact strings.

Interview question

Why did you use embeddings for duplicate detection?

Answer:

“Exact string matching cannot reliably detect duplicate complaints when citizens use different wording. I generate embeddings for complaint descriptions and compare them using cosine distance. This allows CivicFlow-AI to detect semantically similar incidents even when the text isn’t identical.”

10. Similarity Threshold

Your threshold:

```
python  
  
0.20
```

means:

```
text  
  
distance <= 0.20 → duplicate  
distance > 0.20 → not duplicate
```

You tested this successfully.

One result was:

```
text  
  
distance = 0.008556  
threshold = 0.20  
is_duplicate = True
```

Another result was:

```
text  
  
distance = 0.0  
threshold = 0.20  
is_duplicate = True
```

You also tested the Street Lights example:

```
text  
  
distance = 0.199526...  
threshold = 0.20
```

and it was classified as duplicate.

Interview question

What does the similarity threshold represent?

Answer:

“The threshold defines how close two embedding vectors need to be before CivicFlow-AI considers them semantically similar enough to represent the same incident. In my current implementation, a

cosine distance of 0.20 or lower is treated as a duplicate."

11. Duplicate Prevention

Detection alone isn't enough.

You connected the detection result to report creation.

Your pipeline:

```
python

duplicate_result = check_duplicate_report(
    db=db,
    embedding=embedding
)
```

Then:

```
python

if duplicate_result["is_duplicate"]:
```

you reject the new report.

12. HTTP 409 Conflict

You used:

```
python

raise HTTPException(
    status_code=409,
    detail={
        "message": "Similar report already exists.",
        ...
    }
)
```

Why 409?

HTTP 409 represents a conflict with the current state of the resource.

In your case:

```
text

User wants to create report
    ↓
Similar existing incident already exists
    ↓
Conflict
    ↓
409
```

Your successful test returned:

```
text

409 Conflict
```

with information such as:

```
json

{
  "message": "Similar report already exists.",
  "report_id": 11,
  "title": "Major Water Pipeline Burst",
  "distance": 0.008556,
  "threshold": 0.2
}
```

Interview answer

“I return HTTP 409 Conflict when a semantically similar report already exists because the requested creation conflicts with the current incident state.”

13. Service Orchestration

You introduced:

```
python

process_report()
```

Your architecture became:

```
text
```

```
create_report()
  ↓
process_report()
  ├── AI classification
  ├── Priority calculation
  ├── Embedding generation
  ├── Duplicate detection
  └── Database persistence
```

This is called **service orchestration**.

14. Why Not Put Everything in the Router?

Your router should primarily deal with HTTP concerns.

Your router:

```
python

@router.post("/reports")
def create_report_endpoint(...):
```

calls:

```
python

create_report(...)
```

The actual business workflow happens inside the service layer.

Interview question

Why keep the router thin?

Answer:

"The router should mainly handle HTTP concerns such as request validation, dependency injection, and responses. Business logic belongs in the service layer. This keeps the application maintainable and makes the business logic easier to test independently."

15. Final `process\_report()` Flow

Your process is:

```
text
process_report()
├── classify_report(description)
├── calculate_priority(ai_result.severity)
├── generate_embedding(description)
├── check_duplicate_report(...)
├── if duplicate → HTTP 409
├── create Report
│   ├── category
│   ├── severity
│   ├── priority
│   ├── summary
│   └── embedding
```

↓
PostgreSQL

16. Retry Logic

You encountered repeated Gemini:

```
text
503 UNAVAILABLE
```

because the model temporarily experienced high demand.

Instead of immediately failing, you added retry logic.

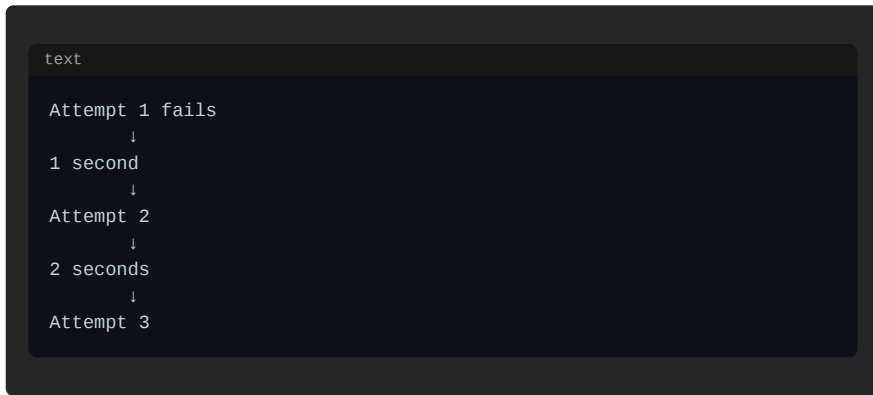
Your classification service uses:

```
python
max_retries = 3
```

and:

```
python
wait_time = 2 ** attempt
```

So the waits are:

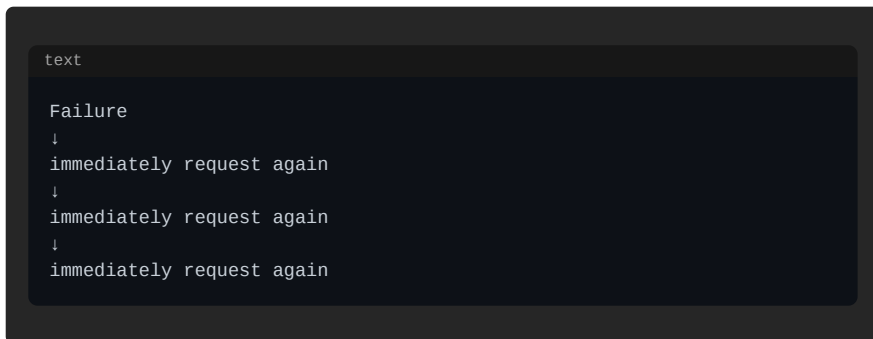


This is called **exponential backoff**.

17. Why Exponential Backoff?

External services can experience temporary failures.

Instead of:



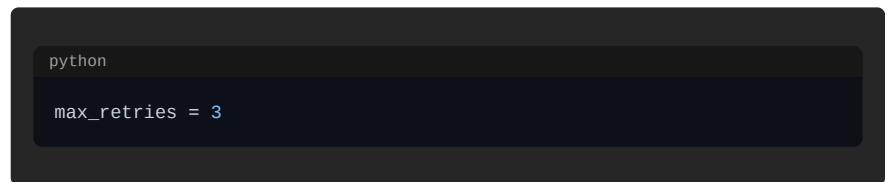
you give the service time to recover.

Interview answer

"I use limited retries with exponential backoff for transient AI-provider failures. The waiting period increases between attempts, reducing repeated immediate requests and giving the external service time to recover."

18. Why Limit Retries?

You used:



because infinite retries are dangerous.

Without a limit:



could leave the request hanging indefinitely.

With three attempts:

```
text

Attempt 1
Attempt 2
Attempt 3
↓
final failure
↓
503
```

19. HTTP 503 for AI Provider Failure

You learned an important distinction:

```
text

409 → duplicate conflict
503 → external AI service unavailable
```

Your AI service eventually returns:

```
python

HTTPException(
    status_code=503,
    detail="AI service is temporarily unavailable..."
)
```

Interview answer

"I return 503 Service Unavailable when the external AI dependency remains unavailable after the retry attempts. This distinguishes provider availability problems from client validation errors or duplicate conflicts."

20. RAG — Retrieval-Augmented Generation

Your Day 3 RAG implementation became part of Day 4 Incident Intelligence.

RAG means:

```
text

Retrieval
+
Augmented context
+
Generation
```

Your implementation:

```

text

Question
↓
Embedding
↓
Retrieve relevant reports
↓
Build context
↓
Gemini
↓
Answer

```

21. RAG Retrieval

Your function:

```

python

retrieve_relevant_reports()

```

does:

```

text

Query embedding
↓
pgvector
↓
Cosine distance

```

```

↓
Closest reports

```

You retrieve more candidates first:

```

python

candidate_limit = limit * 3

```

Then remove exact duplicate descriptions.

This gives your RAG system cleaner context.

22. Why Retrieve More Candidates?

You used:

```

python

candidate_limit = limit * 3

```

If you need:

```

text

limit = 5

```

you initially retrieve:

```
text

15 candidates
```

Then you remove exact duplicate descriptions and keep the required number of unique results.

This improves the chance of getting enough unique contextual reports.

23. Excluding the Current Report

When generating intelligence for an existing report, you added:

```
python

exclude_report_id
```

and:

```
python

query = query.where(
    Report.id != exclude_report_id
)
```

Why?

Without exclusion:

```
text

Report 11
↓
embedding
↓
search
↓
Report 11
distance = 0.0
```

The report would retrieve itself.

For incident intelligence, you want **related existing incidents**, not the current report itself.

24. RAG Context

Your `build_rag_context()` produces information such as:

```
text

Report ID
Title
Description
Category
Severity
Priority
Similarity distance
```

Example:

```
text

Report ID: 12
Title: Water Pipeline Issue Again
Description: A water pipeline has burst...
Category: Water
Severity: High
Priority: HIGH
Similarity distance: 0.0123
```

This becomes the context given to Gemini.

25. Grounded Generation

Your RAG prompt tells Gemini:

```
text

Use ONLY the CivicFlow report information
provided in the context.
```

It also instructs the model:

```
text

Do not invent incidents, locations,
severity levels, priorities, or other facts.
```

This is important because the goal is to ground the response in your application's database.

Interview question

How do you reduce hallucination in your RAG system?

Answer:

```
"I retrieve relevant CivicFlow reports and provide them as explicit context to the LLM. My
prompt instructs the model to answer using only that context and not invent incident information.
I also return the retrieved reports as sources so the response can be traced back to the underlying
data."
```

26. Sources

Your Incident Intelligence response contains:

```
json

"sources": [
  {
    "report_id": 7,
    "title": "Broken Water Pipeline",
    "distance": 0.0469
  },
  {
    "report_id": 12,
    "title": "Water Pipeline Issue Again",
    "distance": 0.247...
  }
]
```

This is useful because the user can see which reports contributed to the answer.

27. Incident Intelligence Service

You created:

```
text
incident_intelligence_service.py
```

Its responsibility is to coordinate:

```
text
Existing Report
  ↓
Embedding
  ↓
Relevant Reports
  ↓
RAG Context
  ↓
Gemini
  ↓
Answer
  ↓
Sources
```

This keeps RAG orchestration separate from CRUD.

28. Report Processing vs RAG

This distinction is **very important** for interviews.

Report Processing

```
text
Complaint
  ↓
Classification
  ↓
Priority
  ↓
Embedding
  ↓
Duplicate Detection
  ↓
Database
```

Incident Intelligence

```
text
Question / Report
  ↓
Embedding
  ↓
Retrieval
  ↓
Context
  ↓
LLM
  ↓
Answer + Sources
```

Interview question

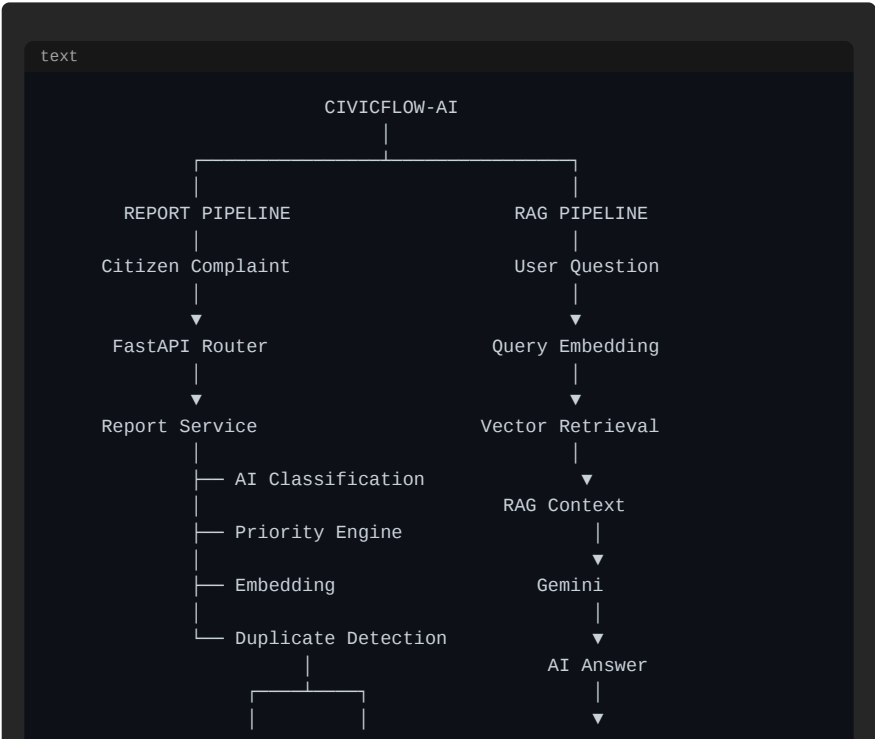
Why didn't you put RAG directly inside `create\_report()`?

Answer:

"Report creation and RAG are different workflows. Report creation is an ingestion pipeline, while RAG is a query-time intelligence workflow. Keeping them separate follows separation of concerns and prevents the report creation process from becoming tightly coupled to question answering."

29. Complete Day 4 Architecture

Memorize this:



Duplicate	New	Sources
409	PostgreSQL	

30. Complete Request Flow — New Report

Suppose citizen submits:

```
json
{
  "title": "Major Water Pipeline Burst",
  "description": "A major water pipeline has burst near Ward 5 and water",
  "location": "Ward 5"
}
```

The system performs:

```
text
1. FastAPI receives request
  ↓
2. Pydantic validates input
  ↓
3. classify_report()
  ↓
4. Gemini returns structured classification
  ↓
5. calculate_priority()
  ↓
6. generate_embedding()
  ↓
```

```

7. check_duplicate_report()
   ↓
8. If duplicate → 409
   ↓
9. Otherwise create Report
   ↓
10. PostgreSQL

```

31. Complete Request Flow — Duplicate

```

text

New complaint
  ↓
AI classification
  ↓
Embedding
  ↓
Similarity search
  ↓
Existing report distance = 0.0085
  ↓
0.0085 <= 0.20
  ↓
Duplicate
  ↓
HTTP 409
  ↓
No database insertion

```

This was successfully tested.

32. Complete Request Flow — Incident Intelligence

For:

```

text

GET /api/reports/11/intelligence

```

your system performs:

```

text

1. Find Report 11
   ↓
2. Generate embedding
   ↓
3. Search similar reports
   ↓
4. Exclude Report 11 itself
   ↓
5. Remove duplicate descriptions
   ↓
6. Build RAG context
   ↓
7. Send context + question to Gemini
   ↓
8. Generate grounded answer
   ↓
9. Return sources

```

You successfully received:

```

text

```

```
200 OK
```

with:

```
text

report_id
title
category
severity
priority
answer
sources
```

🌈 That proves the full Incident Intelligence pipeline works.

33. Error Handling Summary

You now have several meaningful HTTP responses:

Situation	Response
Valid report	`200` / configured success response
Invalid Pydantic input	`422`
Report doesn't exist	`404`
Similar report exists	`409`
Gemini unavailable	`503`
Unexpected unhandled error	`500`

Interview question

Why is error handling important in your project?

“CivicFlow-AI depends on external AI services and database operations, so failures need to be communicated clearly. I use appropriate HTTP status codes to distinguish validation errors, missing resources, duplicate conflicts, and temporary AI-provider failures.”

34. Separation of Responsibilities

Memorize this table.

File	Responsibility
`report_router.py`	HTTP/API layer
`report_service.py`	Report business workflow
`ai_service.py`	Gemini classification
`priority_service.py`	Priority business rules
`embedding_service.py`	Generate embeddings
`similarity_service.py`	Semantic duplicate detection
`rag_service.py`	Retrieval + context
`rag_ai_service.py`	RAG LLM generation
`incident_intelligence_service.py`	Incident intelligence orchestration
`report_model.py`	Database model
`report_schema.py`	API validation/serialization

This is an excellent interview topic.

35. Most Important Interview Questions

Q1. Explain your CivicFlow-AI project.

“CivicFlow-AI is an intelligent civic incident management system. Citizens can submit complaints, and the backend uses AI to classify the incident, determine severity and summary, calculate operational priority, generate embeddings, and detect semantically duplicate reports. PostgreSQL with pgvector stores the incident data and embeddings. The system also uses RAG to retrieve relevant existing incidents and generate grounded AI responses with source reports.”

Q2. Why FastAPI?

“FastAPI provides a lightweight Python framework for building REST APIs, automatic request validation through Pydantic, dependency injection, and automatic OpenAPI/Swagger documentation.”

Q3. What is Pydantic doing?

“Pydantic validates incoming API data against defined schemas before the business logic executes.”

Q4. What is the role of Gemini?

“Gemini performs natural-language classification and generation tasks. In CivicFlow-AI, it classifies complaints into categories and severity and also generates grounded answers for the RAG workflow.”

Q5. What is an embedding?

“An embedding is a numerical vector representation of text that captures semantic information. Similar meanings tend to have similar vector representations.”

Q6. Why pgvector?

“pgvector allows PostgreSQL to store and query vector embeddings alongside normal relational data. This lets CivicFlow-AI perform semantic similarity searches without introducing a separate vector database.”

Q7. Why cosine distance?

“Cosine distance measures the difference in direction between vectors. In our implementation, a smaller cosine distance indicates greater semantic similarity.”

Q8. How do you detect duplicates?

“I generate an embedding for the incoming complaint, compare it against stored report embeddings using cosine distance, retrieve the closest report, and compare the distance against a config-”

ured threshold. If the distance is at or below the threshold, I reject the new report with HTTP 409.""

Q9. Why not exact matching?

""Exact matching only detects identical text. Embedding-based semantic similarity can identify different sentences that describe the same incident.""

Q10. Why use a separate priority engine?

""Priority is a deterministic business rule. I don't want operational priority to depend entirely on probabilistic LLM output. I use AI-generated severity and apply backend rules to calculate the final priority.""

Q11. What is RAG?

""Retrieval-Augmented Generation combines information retrieval with language generation. CivicFlow-AI retrieves relevant incident reports from its database and provides them as context to Gemini so the generated answer is grounded in the application's data.""

Q12. How do you reduce hallucination?

""I retrieve relevant reports, explicitly provide them as context, instruct the LLM to use only that context, and return the retrieved reports as sources.""

Q13. Why exclude the current report from RAG retrieval?

""When generating intelligence for an existing report, retrieving the same report would produce a zero-distance match and doesn't provide useful related-incident information. Therefore I exclude the current report ID.""

Q14. What happens if Gemini is unavailable?

""The AI service retries the request with exponential backoff for a limited number of attempts. If the provider remains unavailable, the API returns HTTP 503 rather than treating the failure as a normal successful request.""

Q15. Why use 409 for duplicates?

""Because the requested resource creation conflicts with an already existing semantically similar incident.""

36. Important Keywords for Your Resume/Interview

You can confidently mention these because you actually implemented them:

text

```

FastAPI
Python
Pydantic
SQLAlchemy
PostgreSQL
pgvector
Gemini API
LLM
Prompt Engineering
Structured AI Output
Embeddings
Cosine Similarity
Semantic Search
Duplicate Detection
RAG
Retrieval-Augmented Generation
Vector Search
Service Layer Architecture
Service Orchestration
REST API
HTTP Status Codes
Exception Handling
Retry Logic
Exponential Backoff

```

37. Day 4 One-Minute Revision

Before an interview, remember this:

```

text
↓
AI
↓
Classification
↓
Severity
↓
Priority Engine

```

```

↓
Embedding
↓
pgvector
↓
Cosine Distance
↓
Duplicate?
├─ Yes → 409
└─ No  → PostgreSQL
      ↓
      RAG
      ↓
      Retrieval
      ↓
      Context
      ↓
      Gemini
      ↓
      Grounded Answer
      ↓
      Sources

```

The core idea:

“AI understands the complaint, backend rules operationalize it, embeddings understand semantic similarity, pgvector finds related incidents, and RAG uses those incidents to produce grounded intelligence.”



Day 4 Assessment — Do This Before Day 5

Try answering these **without looking at the notes**:

1. What is the difference between **severity** and **priority**?

2. Why did you create a separate `priority_service.py`?
3. Why did you normalize severity using `.upper()`?
4. What is an embedding?
5. Why are embeddings useful for duplicate detection?
6. What does cosine distance mean in your project?
7. Why is `0.0085` considered more similar than `0.25`?
8. What does the `0.20` threshold represent?
9. Why do you return HTTP 409 for duplicate reports?
10. What is service orchestration?
11. Why should the FastAPI router remain thin?
12. What is exponential backoff?
13. Why did you limit retries to 3?
14. Why does Gemini failure result in HTTP 503?
15. What is RAG?
16. Explain your RAG pipeline step-by-step.
17. Why do you exclude the current report from RAG retrieval?
18. How does your system reduce hallucination?
19. Why do you return RAG sources?
20. **Explain the complete CivicFlow-AI architecture from citizen complaint → AI → database → RAG answer.**

★ Most important challenge

Answer **Question 20 in your own words** as if I am the interviewer.

After you answer these, I'll evaluate your Day 4 understanding and then we'll start **Day 5** with the next CivicFlow-AI module.